

Archivos de Bajo Nivel

Los archivos en UNIX son particularmente importantes debido a que proporcionan una interface simple y consistente entre los servicios del sistema operativo y los dispositivos. En UNIX todo es un archivo; esto significa que los programas, en general, utilizan los archivos de disco, puertos seriales, impresoras y cualquier otro dispositivo exactamente de la misma manera que usaría un archivo.

Los dispositivos en UNIX están representados por archivos, algunos de los dispositivos más importantes son los siguientes:

`/dev/console`. Este dispositivo representa la consola del sistema. Los mensajes de error y diagnóstico frecuentemente son enviados a este dispositivo. Cada sistema UNIX tiene una determinada terminal o pantalla para recibir los mensajes de consola.

`/dev/tty`. El archivo especial `/dev/tty` es un alias) para el control de un proceso desde una terminal si este fue ejecutado en una.

`/dev/null`. Este es el dispositivo nulo, toda salida enviada hacia este dispositivo es descartada. Al leer este archivo se envía inmediatamente un fin de archivo. Puede usarse para vaciar archivos utilizando el comando `cp`. Cualquier salida no deseada frecuentemente se redirecciona a `/dev/null`.

Otros dispositivos que se encuentran en el directorio `/dev` son el disco duro y los drives, puertos de comunicaciones, lectoras de cinta, CD-ROM, tarjetas de sonido y dispositivos representando el estado interno del sistema. Para tener acceso a estos archivos necesita permisos de superusuario, los usuarios normales no pueden escribir programas para acceder directamente a dispositivos como el disco duro. Los nombres de dispositivo pueden variar de sistema en sistema, Solaris y Linux tienen aplicaciones que corren como superusuario para manejar los dispositivos los cuales de otra manera serían inaccesibles.

Cada programa en ejecución, llamado proceso, tiene asociado un número de descriptores de archivo, estos son números enteros que pueden usarse para acceder a archivos o dispositivos abiertos. La cantidad de archivos disponibles varía dependiendo de cómo haya sido configurado el sistema. Cuando un programa inicia, este usualmente tiene tres de estos descriptores ya abiertos:

0 entrada estándar.
1 salida estándar.
2 error estándar.

Es posible tener acceso y controlar archivos y dispositivos a través de un reducido número de funciones. Estas funciones conocidas como llamadas al sistema son proporcionadas por Unix y Linux y que son la interface con el sistema operativo.

int **creat**(const char * pathname, int perms)

creat se comporta diferente dependiendo de si el archivo existe o no. Si el archivo no existe, entonces un nuevo i-node es asignado, un i-node contiene la siguiente información: tipo de archivo, número de enlaces, dueño, grupo, permisos, tamaño en bytes, hora de último acceso, hora de última modificación y estatus de cambio; es decir, si el i-node en sí tuvo algún cambio. Después de asignar el i-node, en el directorio donde el archivo será creado se establece un enlace hacia el archivo.

Para poder crear el archivo el usuario o el grupo al que pertenece debe tener permiso de escritura en el directorio especificado. El archivo recibe los permisos que se especifiquen en el argumento permisos.

Bit	Operación	Efectivo para
8	Lectura	Dueño
7	Escritura	Dueño
6	Ejecución	Dueño
5	Lectura	Grupo
4	Escritura	Grupo
3	Ejecución	Grupo
2	Lectura	Otros
1	Escritura	Otros
0	Ejecución	Otros

Los permisos son especificados con un número octal, por ejemplo, si queremos otorgar todos los permisos para el dueño y el grupo, y para otros usuarios sólo permiso de lectura y ejecución

bit	
8	1
7	1
6	1 -----> 7
5	1
4	1
3	1 -----> 7
2	1
1	0
0	1 -----> 5

Entonces tendríamos que asignar al parámetro de permisos un 775.

Si el archivo ya existe no hay cambios en el propietario o en los permisos; pero su longitud se trunca a 0 y el archivo se abre para escritura. La hora de modificación y el estatus de cambio se ponen a la hora actual.

Función Open

Para crear un nuevo descriptor de archivo se puede utilizar la función **open**. La función **creat** es otra opción estandarizada por POSIX y soportada por Linux, pero su uso es casi obsoleto.

Existen dos formatos para abrir un archivo utilizando la función **open**. El primero, utiliza dos parámetros, se utiliza para leer o escribir un archivo ya existente. La forma más nueva, tiene tres argumentos y puede realizar todo lo que hace el formato anterior, además de lo que hace la función **creat**.

Formato con dos parámetros `int open (const char * pathname, int flags)`

El archivo especificado en `pathname` debe existir.

Si `flags` tiene un valor de 0, el archivo se abre para lectura.

Si `flags` tiene 1, el archivo se abre para escritura.

Si `flags` vale 2, el archivo se abre para lectura y escritura.

- La función devuelve un descriptor de archivo, el cual puede utilizarse en otra función como `lseek`, `read`, `write` y `close`.
- El descriptor de archivo es único y no puede ser compartido por cualquier otro proceso que este ejecutándose. Si dos programas tienen un archivo abierto al mismo tiempo, mantienen diferentes descriptors de archivo. Al abrirse el archivo, el puntero se posiciona en el primer byte del archivo.
- Si el archivo se abre para lectura y escritura simultáneamente habrá un sólo puntero; por lo que generalmente un `lseek` precede a una escritura o lectura.

Si la operación no es exitosa la función regresa un valor de `-1`. También modifica el valor de la variable de ambiente **errno**, para indicar la razón de la falla.

Formato con tres parámetros `int open (const char * pathname, int flags, int permissions)`

En el formato de 3 parámetros el argumento `flags` toma valores adicionales a 0,1 y 2

<code>O_RDONLY</code>	Abrir solo para lectura.
<code>O_WRONLY</code>	Abrir solo para escritura.
<code>O_RDWR</code>	Abrir para lectura y escritura
<code>O_CREAT</code>	Si el archivo no existe, lo crea . Es es el único caso donde el tercer argumento es usado.
<code>O_TRUNC</code>	Si el archivo existe, trunca su longitud a cero.
<code>O_EXCL</code>	Si <code>O_CREAT</code> también está habilitado devuelve error si el archivo ya existe.
<code>O_APPEND</code>	Asegura que todas las escrituras subsecuentes ocurran al final del archivo.

Estas constantes simbólicas, pueden combinarse por medio del operador OR (`|`), para alcanzar diferentes efectos, aunque no todas las combinaciones son significativas.

Cuando creamos un archivo utilizando la bandera **`O_CREAT`**, se debe especificar el tercer argumento, el modo, el cual puede formarse a través de una operación OR entre las banderas definidas en la biblioteca **`sys/stat.h`**:

<code>S_IRWXU</code>	Todos los derechos al propietario.
<code>S_IRUSR</code>	Derechos de lectura al propietario.
<code>S_IWUSR</code>	Derechos de escritura al propietario.
<code>S_IXUSR</code>	Derechos de ejecución al propietario.
<code>S_IRWXG</code>	Todos los derechos al grupo.
<code>S_IRGRP</code>	Derechos de lectura al grupo.
<code>S_IWGRP</code>	Derechos de escritura al grupo.
<code>S_IXGRP</code>	Derechos de ejecución al grupo.
<code>S_IRWXO</code>	Todos los derechos a otros.
<code>S_IROTH</code>	Derechos de lectura a otros.
<code>S_IWOTH</code>	Derechos de escritura a otros.
<code>S_IXOTH</code>	Derechos de ejecución a otros.

Existen un par de factores que pueden afectar los permisos de archivo. Primero, los permisos especificados son usados solamente si el archivo está siendo creado. El segundo es la máscara de usuario, especificada a través del comando **`umask`**. Al ejecutar **`open`** se realiza una operación XOR entre el valor del modo especificado en **`open`** y el inverso de la máscara de usuario. Por ejemplo, si la máscara de usuario es 001 y en **`open`** se especifica la bandera **`S_IXOTH`**, el archivo no será creado con el permiso de ejecución para otros, porque la máscara de usuario especifica que el permiso de ejecución para otros no se ha otorgado. Las banderas en **`open`** y **`creat`** son peticiones para asignar permisos. Si los permisos serán o no otorgados depende del valor actual especificado **`umask`**.

umask

El valor de la máscara de permisos se puede cambiar a través del comando **`umask`**. La máscara es un número octal de 3 dígitos. Cada dígito se refiere a los permisos de dueño, grupo y otros respectivamente. Un dígito se forma con la combinación de los siguientes valores:

0	Otorgar todos los permisos.
4	Permiso de lectura deshabilitado.
2	Denegar permiso de escritura.
1	Denegar permiso de ejecución.

Por ejemplo, para bloquear los derechos de ejecución y escritura al grupo y el derecho de escritura a otros la máscara sería:

Primer dígito	(Dueño)	0	Otorgar todos los permisos.
Segundo dígito	(Grupo)	1+2=3	Denegar permisos de ejecución y escritura.
Tercer dígito	(Otros)	2	Denegar permiso de escritura.
La orden entonces se escribiría:			<code>umask 032</code>

`int write(int descriptor, char * buffer, int nbytes)`

write escribe los nbytes contenidos en buffer a el archivo representado por el descriptor. La escritura inicia en la posición actual del puntero del archivo y después de la escritura se incrementa en los nbytes escritos. La función regresa un -1 si hubo algún error y no se pudo escribir, si la operación fue exitosa entonces regresa el número de bytes escritos.

int **read**(int descriptor, char * buffer, int nbytes)

La función read lee nbytes del archivo y los guarda en el buffer. La lectura inicia en la posición actual del puntero de archivo. Una vez realizada la lectura el puntero se incrementa en los nbytes leídos. La función read regresa la cantidad de bytes leídos. Si solo se pudo leer un fin de archivo la función regresa 0 y un -1 si hubo un error y no se pudo realizar la lectura.

int **close** (int descriptor)

Lo más importante acerca de la función close es que prácticamente no hace nada. No vacía ningún buffer, lo único que hace es poner disponible el descriptor para reusarlo. De hecho, si el descriptor de archivo no se necesita otra vez, no existe la necesidad de ejecutar close. El archivo se cerrará cuando el proceso termine.

long **lseek** (int descriptor, long offset, int referencia)

La función lseek posiciona el apuntador de archivo para la siguiente lectura o escritura, esto es, se puede utilizar para ubicar el apuntador de archivo en la posición donde ocurrirá la siguiente operación. El apuntador de archivo puede ubicarse en una posición absoluta, o en una posición relativa a la posición actual o al final del archivo. El parámetro offset se usa para especificar el desplazamiento del apuntador y el parámetro referencia especifica como se usará el offset. El parámetro referencia puede ser uno de los siguientes:

SEEK_SET Offset es una posición absoluta.

SEEK_CUR Offset es relativo a la posición actual.

SEEK_END Offset es relativo al final del archivo.

lseek regresa el desplazamiento medido en bytes desde el inicio del archivo hasta la ubicación actual del puntero, o regresa -1 si la operación no tuvo éxito. La función lseek forma parte de la biblioteca **/types.h**

Ejemplo No. 1 create

create: El nombre del archivo se recibe desde la línea de comando. El archivo será creado con todos los permisos para el dueño y ninguno para el grupo ni para otros.

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<fcntl.h>

int main(int totarg, char *arg[])
{ int descriptor;
  if (totarg!=2)
  { printf("Numero de parametros invalido");
    exit(0);
  }
  descriptor=creat(arg[1],0700);
  if (descriptor==0)
    printf("El archivo no se pudo crear");
  else
    printf("El archivo %s tiene el descriptor %d", arg[1], descriptor);

  return(0);
}
```

Ejemplo No. 2 open-write

El programa crea un archivo llamado num.txt y graba los números que el usuario indique.

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<fcntl.h>
#include<string.h>

int main()
{ int descriptor;
  int num,grabados;
  char buffer [255],otro;

  descriptor=open("num.txt",O_WRONLY|O_CREAT,0700);
  if (descriptor==0)
    printf("El archivo no se pudo crear");
  else
  {
    printf("El archivo tiene el descriptor %d", descriptor);
do
  {
    printf("\n escriba un numero:");
    scanf("%d",&num);
    sprintf(buffer,"Numero: %d",num);
    puts(buffer);
    grabados=write(descriptor,buffer,strlen(buffer) );
    printf("%d bytes grabados, \nDesea grabar otra numero", grabados);
    fflush(stdin);
    scanf("%c",&otro);
  } while(otro!='n');
  close(descriptor);
}
return(0);
}
```

Ejemplo No. 3 open-read.

El programa lee el contenido de un archivo y lo muestra en pantalla.

```
#include<stdio.h>
#include<unistd.h>
#include<fcntl.h>
#include<string.h>
int main()
{ int descriptor,leidos, cont=0;
  char carac, cad[25];

  descriptor=open("ciudades.txt",O_RDONLY);
  if (descriptor==0)
    printf("El archivo no se pudo abrir");
  do
  { cont=0;
    cad[0]='\x0';
    do
    { leidos=read(descriptor,&carac,1);
      cad[cont]=carac;
      printf("%c\n",cad[cont]);
      cont++;

    } while(leidos!=0 && carac!='\n' );
    cad[cont]='\x0';
    printf("%s",cad);

  }while(leidos!=0);
  return(0);
}
```

Ejemplo 4 Lectura con lseek

El programa abre un archivo en modo de solo lectura y lee el contenido de un archivo a partir del byte 19.

```
#include<stdio.h>
#include<unistd.h>
#include<fcntl.h>
#include<string.h>
int main()
{ int descriptor,leidos, cont=0;
  char carac, cad[25];

  descriptor=open("ciudades.txt",O_RDONLY);
  if (descriptor==0)
    printf("El archivo no se pudo abrir");
  else
  { lseek(descriptor,19,SEEK_SET);
    do
    { cont=0;
      cad[0]='\x0';
      do
      { leidos=read(descriptor,&carac,1);
        cad[cont]=carac;
        printf("%c\n",cad[cont]);
        cont++;

      }
      while(leidos!=0 && carac!='\n' );
      cad[cont]='\x0';
      printf("%s",cad);

    }while(leidos!=0);
  }
  return(0);
}
```

Ejemplo No. 5 Edición de archivos con lseek

```
#include<stdio.h>
#include<unistd.h>
#include<fcntl.h>
#include<string.h>
int main()
{ int descriptor,grabados;
  char cad[25]="";

  descriptor=open("ciudades.txt",O_RDWR);
  if (descriptor==0)
    printf("El archivo no se pudo abrir");
  else
  { printf("\n escriba un numero:");
    scanf("%s",cad);
    lseek(descriptor,19,SEEK_SET);
    grabados=write(descriptor,cad,strlen(cad) );
    if (grabados >0)
      printf("Se grabaron %d bytes", grabados);
    else
      printf("No se pudo grabar");
    close(descriptor);
  }
  return(0);
}
```